

Construí un parque jurásico en 3D con Claude Code: lo que funcionó, lo que no y el método que ahora uso

Carolina Rodríguez Barcena · Proyecto: github.com/Estivbi/jurassic-dino



Hace poco publiqué una web que es una especie de parque jurásico: conduces un jeep por un parque con flora del Mesozoico y un lago, entre nueve especies a su tamaño real, bajo un cielo que es el de tu ciudad (Sol, Luna con su fase y estrellas calculados con tu ubicación y tu hora). Cada animal tiene su ficha, un mito del cine desmontado y un quiz. Todo corre en el navegador del móvil, sin instalar nada.

Lo construí con **Claude Code** y, para lo más complejo, **Opus 5.5**. Y no, no salió en cinco minutos: salió porque fui guiando cada paso. Este artículo es lo que me llevo, con la parte técnica incluida, porque creo que lo interesante no es *que* se pueda, sino *cómo* se consigue que salga algo mantenible.

Stack: React 19 + TypeScript + Vite, three.js, Tailwind v4 y Framer Motion para la interfaz, SunCalc para la astronomía, desplegado en Vercel. Sin backend, sin cookies.

1. Lo primero que hizo fue lo que esperaba que no hiciera

Mis expectativas eran más bien un "a ver qué pasa", porque nunca había hecho nada parecido. Y lo que pasó es que el primer resultado fue flojo. Los dinosaurios eran figuras de juguete montadas con formas sencillas, y el entorno quedaba muy lejos de lo que yo tenía en la cabeza. Con Opus 5.5 para el desarrollo más complejo la cosa mejoró mucho, pero la lección no cambió: **la IA hace lo que le pides, no lo que quieres**. Si no eres explícita, no afinas.

El punto de inflexión fue dejar de dibujar los dinosaurios y buscar **modelos 3D reales con licencia libre**. Eso cambió el proyecto, y también abrió un problema que sí es de ingeniería: un modelo bonito en un visor pesa mucho más de lo que un móvil aguanta.

2. Meter modelos 3D en una web móvil

Cada modelo pasa por un proceso fijo, que está documentado en una *skill* del repo para que no dependa de que yo me acuerde:

```
glTF-transform optimize in.glb out.glb \
--texture-size 1024 --texture-compress webp \
--compress meshopt --simplify-ratio 0.3
```

Texturas a 1024 px en WebP, geometría comprimida con Meshopt y simplificación de malla. Así, los nueve modelos suman unos 4,1 MB.

Y aparecieron cosas que no se ven a simple vista:

- **Árboles sin hojas.** Para ahorrar peso, `optimize` elimina atributos que cree que sobran. En un modelo sin texturas borraba las coordenadas UV y las hojas desaparecían. Se arregla con `--prune-attributes false`.
- **Dinosaurios diminutos.** Los modelos con esqueleto vienen en escalas distintas, y calcular su tamaño con la geometría base daba valores absurdos. Hay que **hornear la pose del esqueleto** antes de medir, y además los GLB comprimidos con Meshopt guardan posiciones cuantizadas, así que antes pasan por una función que las convierte a flotantes.
- **Licencias.** Una regla que dejamos escrita: cada modelo necesita licencia verificada y su entrada en los créditos, y **nada de modelos extraídos de videojuegos aunque digan ser CC BY**. Esa parte no la puede decidir la IA por ti.

3. Decidir el stack antes de escribir una línea

Aquí es donde más aporté yo. Mi criterio desde el principio fue **móvil primero** y **no escribir código de más**, y eso se tradujo en decisiones concretas:

Física propia en lugar de un motor. Me preocupaba lo que pesara. Medimos: Rapier (versión web, WebAssembly) ronda **1,6 MB** comprimido; cannon-es, unos 36 KB. Para un jeep sobre un terreno con obstáculos circulares no hacía falta ninguno. La física propia son pocos KB:

```
// Agarre lateral: los neumáticos anulan el deslizamiento de lado
vSide *= Math.exp(-TIRE_GRIP * (1 - 0.6 * slipFactor) * dt)
// Giro tipo Ackermann simplificado
const yawTarget = (vForward / WHEELBASE) * Math.tan(this.steer)
```

Velocidad 2D con inercia, pendientes que frenan o empujan, gravedad (en una cresta rápida el jeep se despega), choques con rebote contra troncos, rocas y dinosaurios con una rejilla espacial, y la carrocería sobre muelles amortiguados.

three.js fuera del bundle inicial. La escena se carga con un `import()` dinámico, para que lo primero que baje el móvil sea la interfaz (unos 119 KB comprimidos) y la escena 3D (unos 195 KB comprimidos) llegue después. Es una regla escrita en el proyecto, no una buena intención.

Calidad adaptativa. El juego mide los FPS en ventanas de 3 segundos y, si caen por debajo de 40, va bajando en escalones: primero la resolución, luego sombras y agua, luego la niebla. Y hay un `?debug=1` con un panel de rendimiento para medir en un móvil real. Esa parte sigue en mi lista, porque es justo lo que la IA no puede hacer.



4. Los bugs: cada uno acabó siendo una regla

Aquí está la parte más útil del proceso. Los bugs de este proyecto los introdujo la IA y los encontró después, no a la primera. Tres ejemplos:

El lago que no se excavaba. La cuenca del lago se calculaba con `THREE.MathUtils.smoothstep` pasando los bordes al revés (`a > b`). En three.js eso devuelve 0, no lo que haría GLSL, y el terreno quedaba plano bajo el agua. Solución:

```
export function smooth(x: number, a: number, b: number): number {  
  const t = Math.min(1, Math.max(0, (x - a) / (b - a)))  
  return t * t * (3 - 2 * t)  
}
```

El agua negra. En calidad alta el lago se veía completamente negro. La clase `Water` de three.js calcula el plano del espejo a partir del eje +Z local del objeto. Si tumbas el plano *dentro de la geometría*, el reflejo apunta a otro sitio. Hay que rotar el objeto (`water.rotation.x = -Math.PI / 2`) y no la geometría.

La Luna. La fase no se calcula con una tabla: la esfera lunar se ilumina desde la dirección real del Sol, y la fase sale sola.

```
float lit = smoothstep(-0.03, 0.12, dot(normalize(vWorldNormal), uSunDir));
```



Lo importante es lo que viene después. Cada una de estas trampas **se escribe en el** `CLAUDE.md`, para que la próxima sesión no vuelva a caer en ella:

Trampas conocidas

- ``THREE.MathUtils.smoothstep`` devuelve 0 con bordes invertidos (``a > b``): usa ``smooth()``.
- SunCalc v2 da ángulos ****en grados**** y el acimut desde el norte en sentido horario.
- Al optimizar un GLB sin texturas, ``gltf-transform optimize`` borra las UV: usa ``--prune-attributes false``.
- Cambiar el número de luces recompila todos los shaders: los faros nunca se ocultan, se ponen a intensidad 0.

Un modelo no recuerda de una sesión a otra. El `CLAUDE.md` es su memoria, y la escribes tú.

5. El método: lo que más me ha enseñado

Si tuviera que quedarme con una sola cosa: **la diferencia no la hace el prompt, la hace el proceso.**

- `CLAUDE.md` y `AGENTS.md`. Normas del proyecto, comandos, arquitectura, trampas conocidas y reglas de trabajo. Los reutilizo como plantilla en proyectos nuevos.
- **Definir la arquitectura antes de empezar**, y decidir qué agentes necesito, cuántos y qué hace cada uno.
- **Skills propias** para lo que se repite: hacer capturas con Playwright y revisar el resultado, añadir un modelo 3D con su licencia, revisar el rigor de los textos educativos y llevar una PR.
- **Dividir en hitos o fases**. Así no gasto la suscripción de golpe y puedo guiar y supervisar cada paso en vez de revisar una montaña de cambios al final.
- **Una PR por tema**, que reviso y fusiono yo. La rama principal está protegida: la IA nunca sube directamente a `main`.
- **Antes de cada push**: typecheck, lint y build sin errores. Está escrito en el `CLAUDE.md`.

- **Control de autoría.** Al principio los commits salían firmados por la IA. Lo resolví en la configuración del repositorio, no pidiéndolo en cada prompt:

```
{  
  "attribution": { "commit": "", "pr": "", "sessionUrl": false },  
  "env": {  
    "GIT_AUTHOR_NAME": "Carolina Rodriguez Barcena",  
    "GIT_COMMITTER_NAME": "Carolina Rodriguez Barcena"  
  }  
}
```

Creo que esto es lo que se llama *spec-driven development*: primero defines qué quieres construir y bajo qué reglas, y después dejas que la IA ejecute. En mi experiencia es lo que separa un resultado que se puede mantener de uno que se te va de las manos.

6. Lo que la IA no puede hacer

- **Probar en un móvil real.** Trabajaba en un entorno sin tarjeta gráfica, con el render por software a 4–6 FPS, y no podía juzgar la fluidez. Por eso pasamos un `?debug=1` para medir yo en el teléfono.
- **Oír.** Probamos voces para los dinosaurios. Nadie sabe cómo sonaban, y a mí me sonaban todas igual y nada a dinosaurio, así que **las quité** y dejé solo el motor del jeep y el choque. Quitar lo que "funciona" pero no suma es una decisión de producto.
- **Saber qué es "bueno".** Eso lo pone quien dirige.

7. Lo que sí hizo muy bien

Para ser justa: en lo que yo no domino, ahorra semanas. Las matrices de astronomía que llevan las estrellas del catálogo al cielo local según la hora sidérea, los shaders de deformación para el andar de los dinosaurios o las particularidades de glTF me habrían costado una barbaridad de documentación. Y las pruebas: capturas automáticas de cada dinosaurio, de día y de noche, y pruebas de conducción. Eso da una primera revisión antes de que yo mire nada.

El proyecto está en github.com/Estivbi/jurassic-dino.